Universidad Galileo	**Fecha 10/10/2026**
Facultad: FISSIC	**Alumno: Pablo Urbina**
Curso: Microprocesadores	**Carnet: 24001508**
Sección: A	**Hora de Laboratorio: 1:00 PM**
Auxiliar: **Brandon Eduardo Mendoza Espinoza**	**Día de Laboratorio: Lunes**

Laboratorio #7 Puertos Paralelos en Lenguaje Ensamblador

Objetivos:

Aprender a desarrollar programas en ensamblador para el microcontrolador, utilizando Visual Studio.

Resumen:

Se implementaron las rutinas LEDs_Init y LED_Toggle en el lenguaje ensamblador en base a un poco de código que se nos proveyó en el material de la práctica, LEDs_init prepara el LED para que su estado cambie, configurando su pin como salida digital, mientras que LED_Toggle cambia el estado del LED cada vez que se llama.

Teoría:

El STM32G431 utiliza un procesador Arm Cortex-M4. Este ejecuta instrucciones Thumb de 16 y 32 bits, lo que permite escribir partes de una aplicación en lenguaje ensamblador y llamarlas desde un programa en C. Los pines de propósito general o GPIO se agrupan en puertos.

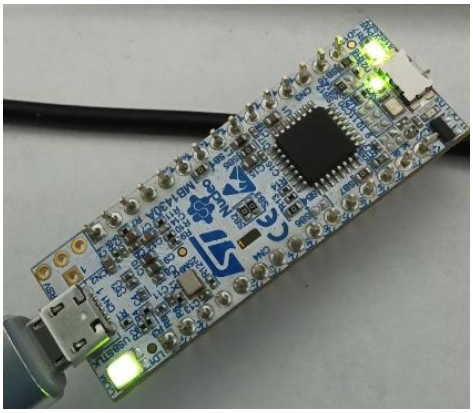
Cada pin puede configurarse de forma individual como entrada, salida, función alternativa o entrada analógica. El registro GPIOx_MODER selecciona ese modo, GPIOx_IDR permite consultar el nivel de los pines, GPIOx_ODR contiene los valores escritos para las salidas y GPIOx_PUPDR configura resistencias internas de pull-up o pull-down cuando es necesario. Para modificar un solo pin sin alterar los demás, se lee el registro, se cambia únicamente el campo o bit correspondiente y se escribe el resultado.

Antes de utilizar un puerto GPIO, debe habilitarse su reloj mediante el registro de control correspondiente del módulo RCC. Para controlar un LED, el programa habilita el reloj del puerto, configura su pin como salida y establece un valor inicial. Después puede invertir el bit de ese pin en ODR.

El temporizador del sistema, SysTick, puede generar interrupciones a intervalos regulares. SysTick_Config recibe la cantidad de ciclos de reloj entre interrupciones, calculada a partir de SystemCoreClock, cuando ocurre una interrupción, se ejecuta SysTick_Handler.

Datos Prácticos:

Video del programa funcionando en la tarjeta (a 50 Hz): https://drive.google.com/file/d/1Wh-RFh4qwN4gLX5tX7cAfntBbHeSr2k9/view?usp=sharing



Respuesta a las preguntas:

¿Por qué las funciones declaradas en main.c utilizan el modificador extern?

Declaran funciones que main.c puede llamar, aunque sus cuerpos están en otro archivo. Durante el enlace, sus nombres se resuelven con los símbolos que GPIO.s hace visibles mediante EXPORT.

¿Dónde están definidas esas subrutinas?

LEDs_Init y LED_Toggle están definidas en GPIO.s, SysTick_Handler está definida en main.c y llama a LED_Toggle.

Conclusiones:

Es posible aprovechar las ventajas tanto de C como de ensamblador en un mismo proyecto, es necesario comprender como se comunican los archivos de C y ensamblador para que funcionen en unión correctamente, también se puede apreciar como código escrito en C se puede traducir sin mayor complicación a código escrito en ensamblador, alcanzando el mismo resultado mientras se tiene mas control sobre como se maneja la memoria y los registros directamente, obteniendo experiencia para saber juzgar cuando es mejor escribir algo de un proyecto en ensamblador y cuando es mejor hacerlo en C.

Bibliografía:

[1] STMicroelectronics, "NUCLEO-G431KB," STMicroelectronics. [En línea]. Disponible en: https://www.st.com/en/evaluation-tools/nucleo-g431kb.html

[2] Arm, "Cortex-M4 Product Support," Arm Support. [En línea]. Disponible en: https://support.arm.com/compute-ip/cortex-m4

[3] STMicroelectronics, "stm32g431xx.h," CMSIS Device G4, GitHub. [En línea]. Disponible en: https://raw.githubusercontent.com/STMicroelectronics/cmsis-device-g4/master/Include/stm32g431xx.h